

Conversion avancée avec PostgreSQL Migrator

Florent JARDIN (DALIBO)

jeudi 8 octobre 2026

Annonce



[Home](#) [About](#) [Download](#) [Documentation](#) [Community](#) [Developers](#) [Support](#) [Donate](#) [Your account](#)



August 13, 2026: [PostgreSQL 18.6, 17.11, 16.15, 15.19, 14.24](#) and 19 Beta 3 Released!

Quick Links

- [About](#)
- [Governance](#)
- [Policies](#)
- [Feature Matrix](#)
- [Donate](#)
- [History](#)

PostgreSQL Migrator 1.0 : first stable release

Posted on **2026-09-10** by Dalibo

 [Related Open Source](#)

Paris, 7th september 2026.

The Dalibo team is pleased to announce the release of **PostgreSQL Migrator** 1.0 stable, a free and open-source tool designed to help migrate databases from Oracle and MySQL/MariaDB to PostgreSQL.

Key features

- **Easy installation:** pure Go binary with zero proprietary dependencies.
- **Fast, offline inspection:** Extract the full source catalog once, then iterate on conversion offline.
- **Complexity scoring:** Quickly assess and compare the difficulty of migration projects.
- **Interactive web UI:** Browse both the original and converted catalogs, search objects, and identify migration blockers.
- **Schema and data conversion:** Converts datatype and SQL expression, convert column to boolean.
- **Code analysis and transpilation:** Transpiles and analyzes SQL, PL/SQL and routines with the companion **transqlate** tool.
- **High-performance data copy:** Streamed bulk COPY to PostgreSQL (or to files / pipes), with low memory usage and significantly higher throughput than traditional tools in our tests.
- **Hackable & configurable:** Reliable JSON catalog data, TOML configuration, and templating for fine-tuning.

Les fichiers d'un projet

`.env`

- Alimenté par la commande `pg_migrate init`

```
PGMSOURCE=oracle://system:manager@localhost/freepdb1  
PGMTARGET=postgresql://florent@localhost/demo
```

pg_migrate.toml

<https://postgresql-migrator.rtf.d.io/en/latest/references/toml/>

```
Schemas = [ "DEMO" ]
```

[Convert.Rules]

```
Path = "Tables/DEMO.CLIENT/Columns/DATE_CREATION"
```

```
Type = "date"
```

```
Expression = "current_date"
```

[Dump]

```
Objects = [  
    "!Roles"
```

```
]
```

pg_migrate.toml

```
Schemas = ["DEMO"]

[[Convert.Rules]]
Path = "Tables/DEMO.CLIENT/Colum..."
Type = "date"
Expression = "current_date"

[Dump]
Objects = [
    "!Roles"
]
```

```
Schemas:
  - DEMO

Convert:
  Rules:
    - Path: Tables/DEMO.CLIENT/Colum...
      Type: date
      Expression: current_date

Dump:
  Objects:
    - "!Roles"
```

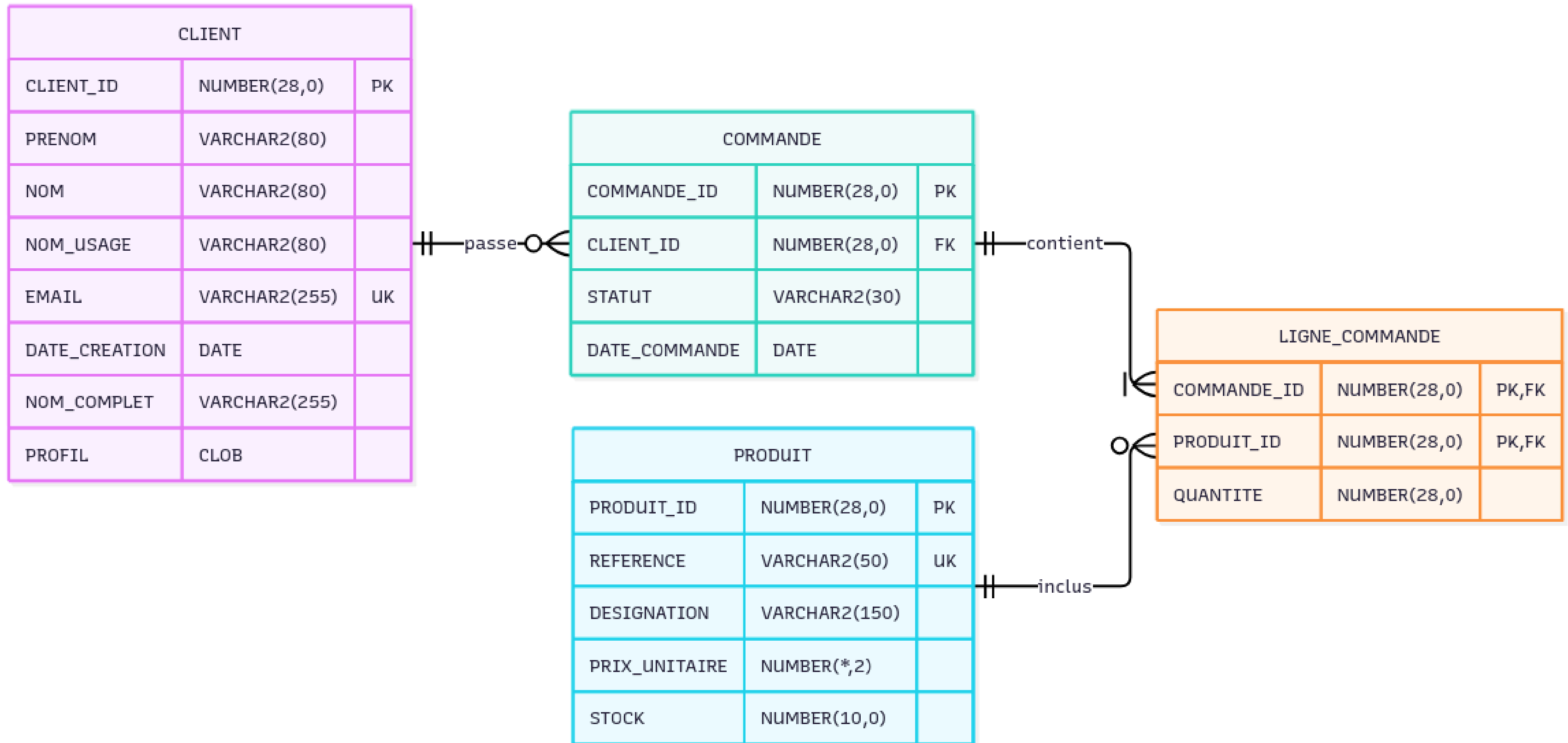
`.pg_migrate/`

- `Source.json` : catalogue source, créé à l'inspection
- `Target.json` : catalogue cible, converti depuis la source
- `Info.json` : état du projet, version de `pg_migrate`, de la source et de la cible
- `Stats.json` : taille des segments, estimation du nombre de lignes par table
- `Map.json` : correspondance des `Path` cibles avec leur source

`pg_migrate.toml` VS. `pg_migrate.auto.toml`

- Comportement similaire au fichier `postgresql.auto.conf`
- Le contenu du fichier `.auto.toml` surcharge celui du `.toml`
- Très utile pour le générateur de configuration (mode expert)

Cas d'usage



Projet de démonstration

- Cloner le dépôt <https://gitlab.com/fljdin/pgm-advanced-conversion>
- Se déplacer dans la branche `main` avec
 - `git move --root` pour partir du premier commit
 - `git move --next` pour remonter la branche

Convert . DataTypes

- Applique une conversion globale pour un type de données
- Complète une liste de correspondance prédéfinie
 - <https://postgresql-migrator.rtf.d.io/en/latest/references/conversion/>

[Convert . DataTypes]

"NUMBER (28, 0) " = "bigint "

Convert .Rules 1/3

- Applique une conversion locale
- Nécessite le chemin (*path*) unique du composant
- Renommer un objet avec *Name*

```
[ [Convert .Rules] ]
```

```
Path = "Tables/DEMO.PRODUIT/Checks/CK_PRODUIT_PRIX_positive"
```

```
Name = "ck_produit_prix_positive"
```

Convert .Rules 2/3

- Changer le type d'une colonne avec `Type`
- Exemple de conversion
 - Correspondance en `jsonb`
 - Exclusion de la contrainte `CHECK (ERREURS IS JSON)`

```
[ [Convert .Rules] ]
```

```
Path = "Tables/DEMO.CLIENT/Columns/PROFIL"
```

```
Type = "jsonb"
```

```
[Dump]
```

```
Objects = [
```

```
    "!Tables/demo.client/Checks/chk_client_profil"
```

```
]
```

Convert .Rules 3/3

- Surcharger une expression avec `Expression`
- Valable pour les valeurs `DEFAULT` et les colonnes virtuelles

```
[[Convert.Rules]]  
Path = "Tables/DEMO.CLIENT/Columns/NOM_COMPLET"  
# TRIM("PRENOM" || ' ' || "NOM" || NVL2("NOM_USAGE", ' (' || "NOM_USAGE" || ') ', NULL))  
Expression = ''  
TRIM(  
    "prenom" || ' ' || "nom"  
    || CASE WHEN "nom_usage" IS NULL  
        THEN ''  
        ELSE ' (' || "nom_usage" || ') '  
    END  
)  
''
```

RenameConstraints et RenameIndexes

- Génère un nom standard pour l'une des catégories
- Renommage systématique
 - des noms générés
 - des noms en conflit avec les tables

```
# Enable automatic renaming of all constraints.  
RenameConstraints = false
```

```
# Enable automatic renaming of all indexes.  
RenameIndexes = true
```

Mode expert

Générateur de configuration

- Commande `_build` non documentée

```
pg_migrate --verbose _build pg_migrate.auto.toml
```

```
# 15:59:15 DEBUG   Read template.           path=pg_migrate.auto.toml.tmpl
#                                           fields=[Source]
# 15:59:15 INFO     Rendering template.      in=pg_migrate.auto.toml.tmpl
#                                           out=pg_migrate.auto.toml
```

Variables de template

```
# pg_migrate.auto.toml.tmpl
```

```
{{ $schema := (index .Source.Schemas 0).Name }}
```

```
[[Convert.Rules]]
```

```
Path = "Schemas/{{ $schema }}"
```

```
Name = "public"
```

Règles dynamiques

```
{{- range .Source.Tables }}  
{{- range .Columns }}  
{{- if eq .Expression "SYSDATE " }}
```

```
[[Convert.Rules]]
```

```
Path = "{{ .ObjectPath }}"  
Type = "date"  
Expression = "current_date"  
{{- end }}  
{{- end }}  
{{- end }}
```

Conclusion ✨

- La donnée structurée permet de travailler hors ligne
 - diagnostiquer
 - naviguer, manipuler
 - convertir
- Nombreuses règles de conversion pour couvrir les besoins
- ... génération de configuration avec les *templates*

Découvrez nos guides

- <https://dali.bo/wspgm> (français)
- <https://postgresql-migrator.rtf.d.io/en/latest/guides/getstarted/>
- <https://postgresql-migrator.rtf.d.io/en/latest/guides/convert/>
- <https://postgresql-migrator.rtf.d.io/en/latest/guides/dump/>

Questions ?



PostgreSQL Migrator

Modernize your database by moving
to PostgreSQL.